AegisScan
Smart Web Vulnerability Scanner

Software Engineering Final Project Report

Course	Software Engineering
Teaching Assistant	Eng. Rawan El Gendy
Team Members	Omar Nabil, Youssef Ashraf, Mazen Mohamed, Abdelrahman Mohamed, Abdelrahman Shaheen
Project Type	Security-focused web application
Containerization	Docker-ready deployment included
GitHub Repository	https://github.com/omarnabilgaafar/aegisscan-cloud

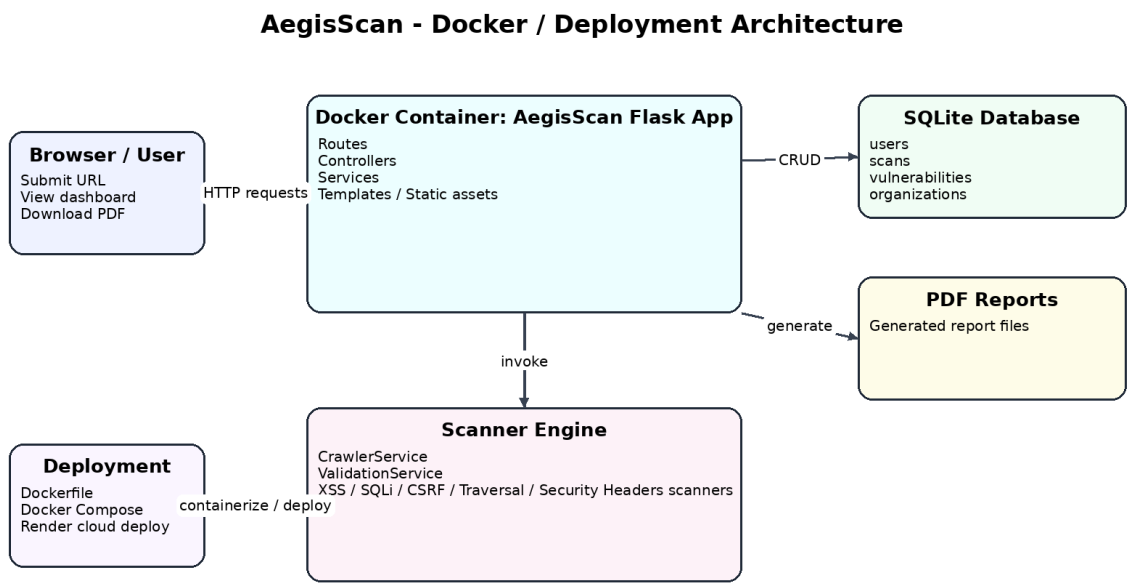


Figure 1. High-level Docker/deployment view of AegisScan.

Executive Summary

AegisScan is a web vulnerability scanning platform designed to help users analyze target web applications, identify common security weaknesses, store findings, and generate downloadable PDF reports. The system combines a Flask-based web interface, modular scanning services, database persistence, and Docker-ready deployment to provide a practical software engineering case study.

The project follows the Software Development Life Cycle (SDLC) requested in the course brief: requirements, design, implementation, testing, and maintenance. The report also includes the required UML diagrams: Use Case, Sequence, Activity, Class, and State Machine.

1. Introduction and Project Scope

1.1 Problem Statement

Manual security checking is slow, repetitive, and error-prone. Students, developers, and small teams often need a lightweight way to inspect a web target for common issues such as cross-site scripting, SQL injection, CSRF weaknesses, directory traversal, and missing security headers.

1.2 Objectives

- Provide a simple interface for entering a target URL and starting a scan.
- Validate the target before execution to keep the educational build controlled and safe.
- Run several security checks and aggregate the results in one scan summary.
- Store scans and vulnerabilities for later viewing in the dashboard.
- Generate printable PDF reports for documentation and review.

1.3 System Scope

In scope: user registration and login, launching scans, viewing scan results, dashboard visualization, report generation, and admin/workspace views. Out of scope: production-grade internet-wide scanning, distributed scan agents, and unrestricted scanning of arbitrary public hosts.

1.4 User Interface Snapshot

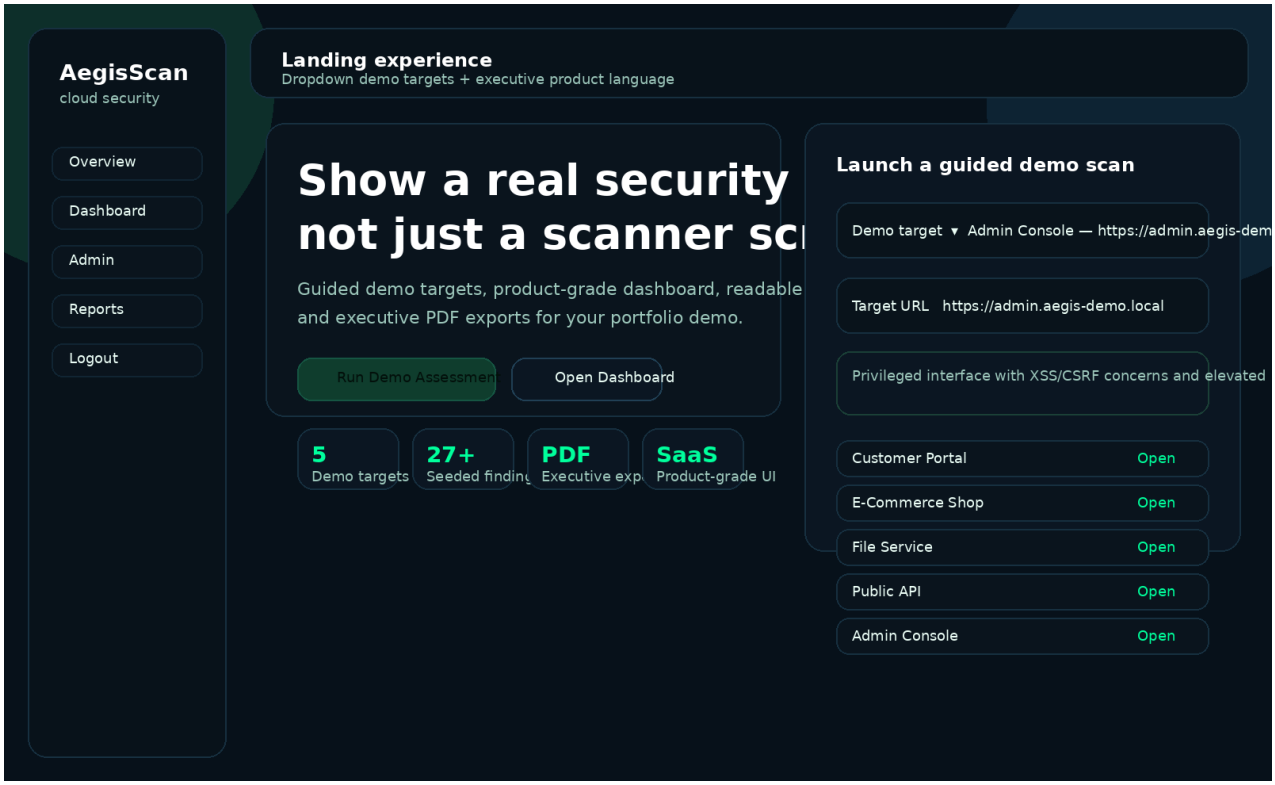


Figure 2. Landing page of the AegisScan application.

2. Requirements Phase

2.1 Functional Requirements

ID	Requirement
FR1	The system shall allow a visitor to register an account and log in.
FR2	The system shall allow a user to submit a target URL for scanning.
FR3	The system shall validate the submitted URL before running the scan.
FR4	The system shall crawl links and forms from the target where applicable.
FR5	The system shall execute security checks for XSS, SQL injection, CSRF, directory traversal, and security headers.
FR6	The system shall aggregate findings, calculate severity counts, and summarize scan risk.
FR7	The system shall save scans and vulnerabilities in the database.
FR8	The system shall display scan results and

	support PDF report download.
FR9	The dashboard shall show recent scans, findings, severity distribution, and workspace usage.
FR10	The admin view shall expose organization and user overview data for administrators.

2.2 Non-Functional Requirements

Attribute	Requirement
Security	Only allowed hosts are scanned in the educational build, and the request must include a CSRF token.
Performance	The system should present a completed result page within an acceptable time for the selected target and show clear feedback in case of limited coverage.
Usability	The web interface should be straightforward for students to use without advanced security knowledge.
Reliability	Scan results and vulnerabilities must be persisted correctly in the database once the scan is completed.
Maintainability	The project uses separated routes, controllers, services, scanners, and models to keep the codebase modular.
Deployability	The application must be container-ready using Docker and support simple cloud deployment.

3. Design Phase

3.1 Architectural Design

The system adopts a layered web architecture. Routes receive HTTP requests, controllers coordinate the request flow, services perform validation, crawling, scanning, and reporting logic, and models represent persisted entities such as users, scans, vulnerabilities, and reports. SQLite is used as the project database in the current build.

Containerization is part of the design. The project includes a Dockerfile, docker-compose configuration, and a Render deployment file. This makes the system easier to demonstrate, deploy, and reproduce on another machine.

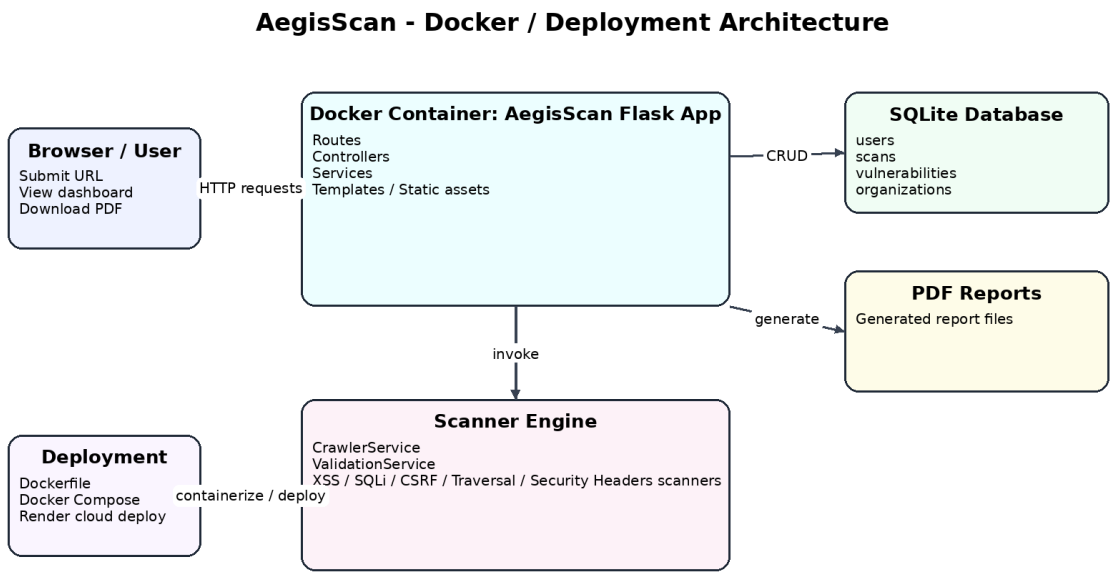


Figure 3. Docker and deployment architecture included in the implementation.

3.2 Design Decisions

- Flask was selected because it is lightweight, clear for academic projects, and fits a modular MVC-like organization.
- Scanning logic was separated into dedicated modules so each vulnerability type can evolve independently.
- Validation was isolated in its own service to enforce safe target rules before scanning.
- PDF report generation was kept as a dedicated reporting module to separate presentation from scan execution.

3.3 Use Case Diagram

Shows the primary actors and high-level system services such as registration, login, scanning, viewing results, downloading reports, dashboard access, and admin management.

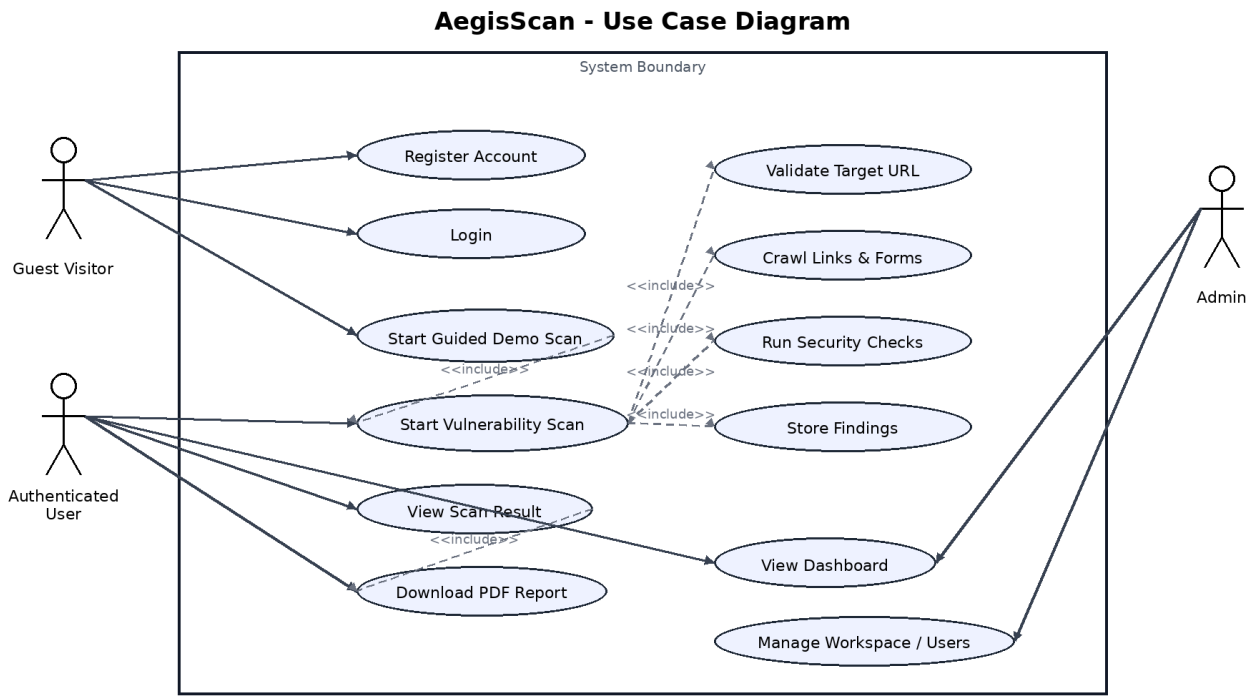


Figure 4. Use Case Diagram for AegisScan.

3.4 Sequence Diagram

Explains the interaction order when a user submits a scan request and the system validates, runs, stores, and returns the result.

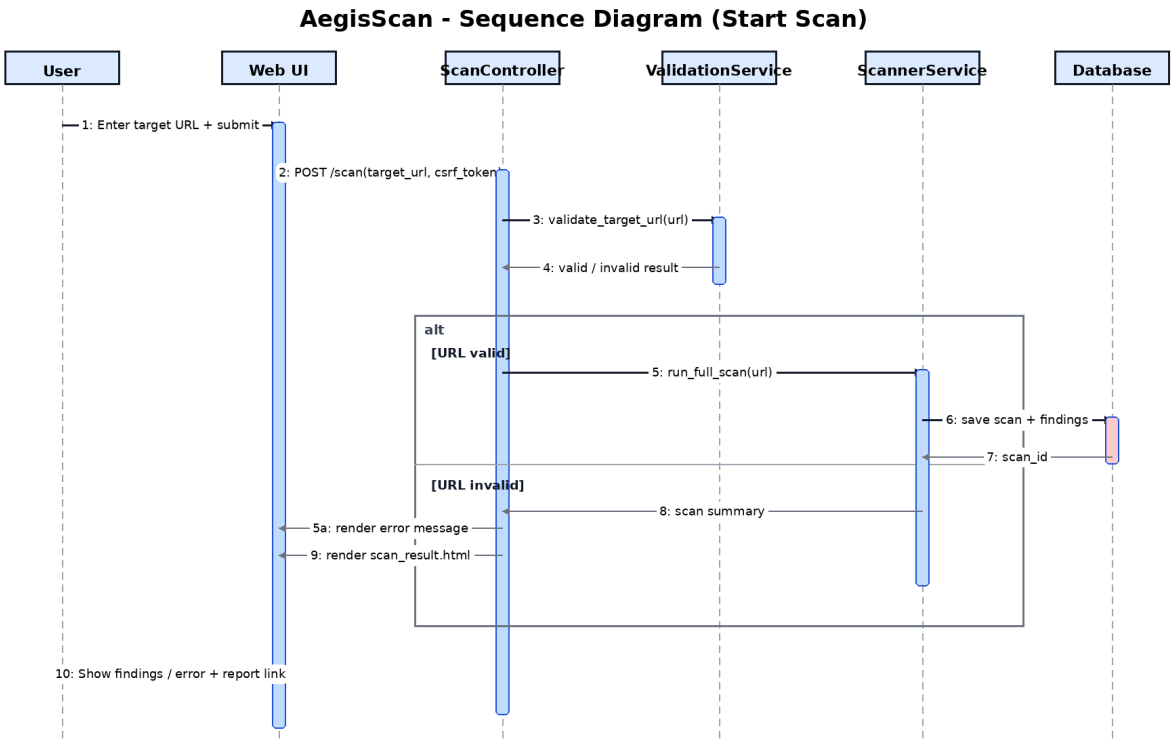


Figure 5. Sequence Diagram for AegisScan.

3.5 Activity Diagram

Represents the workflow from launching the application to validating the URL, running the scan, handling invalid input, and displaying the result.

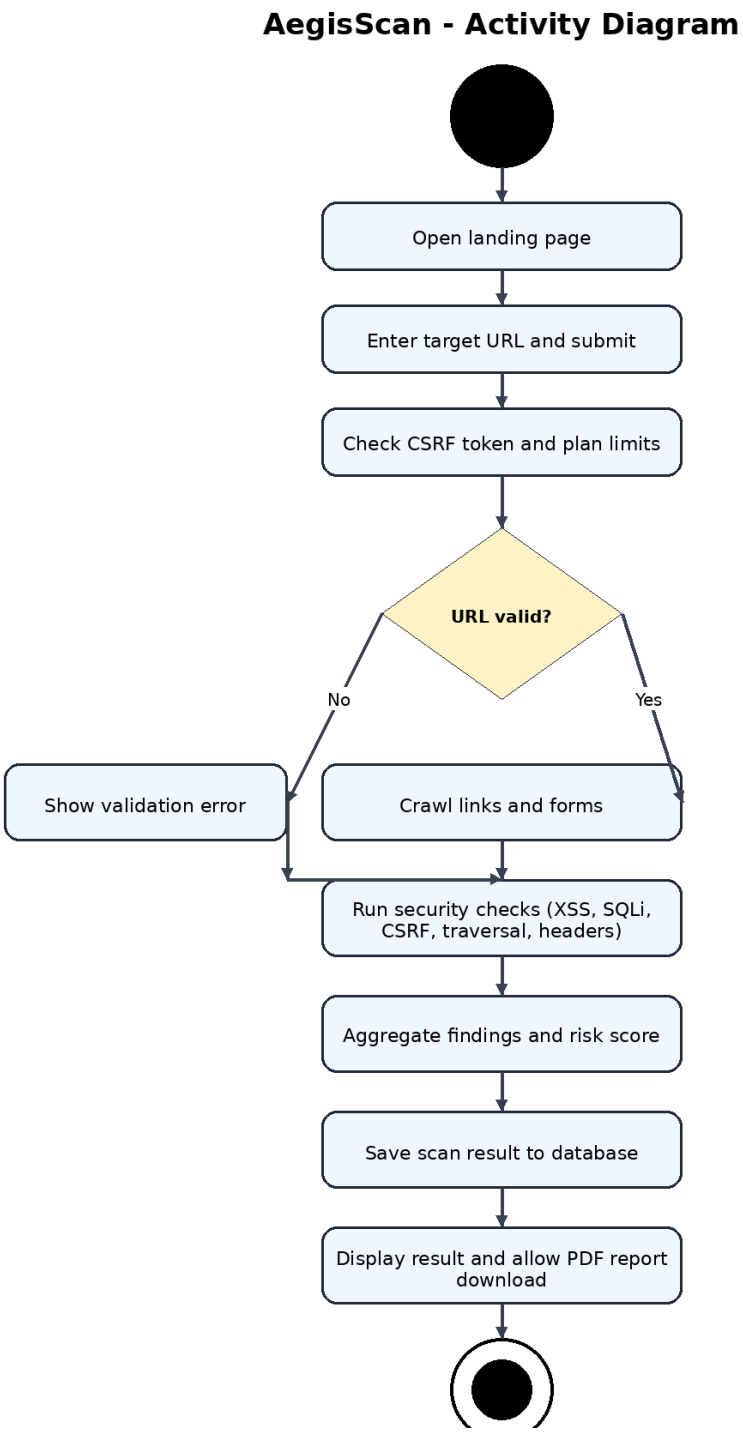


Figure 6. Activity Diagram for AegisScan.

3.6 Class Diagram

Illustrates the core domain objects and service classes used by the project.

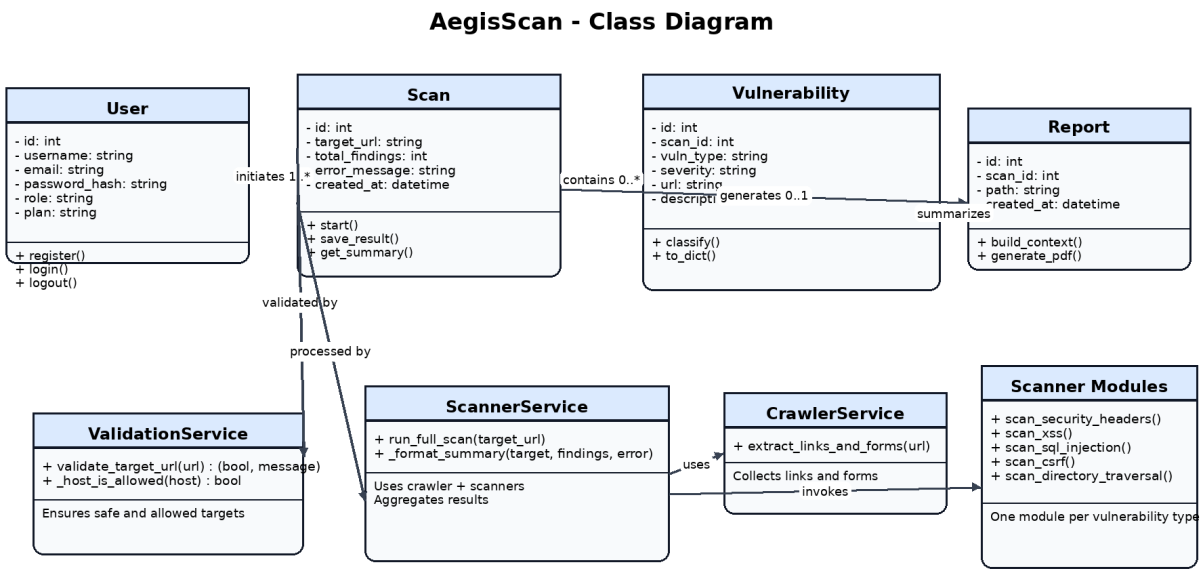


Figure 7. Class Diagram for AegisScan.

3.7 State Machine Diagram

Shows how the scan process changes state from idle to validation, scanning, saving, completion, or failure.

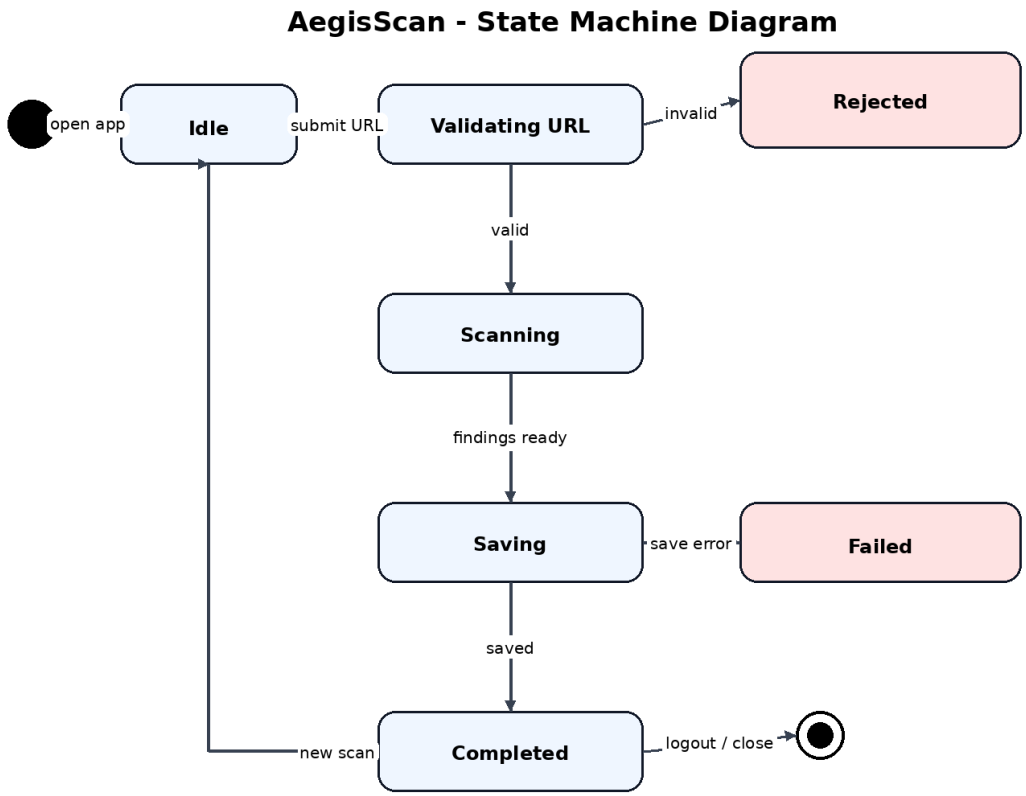


Figure 8. State Machine Diagram for AegisScan.

4. Implementation Phase

4.1 Technologies and Tools

Technology	Purpose
Python + Flask	Web application framework and request handling
SQLite	Local persistence for users, scans, findings, organizations, and keys
HTML/CSS/JS	User interface templates and dashboard behavior
Docker / Docker Compose	Containerization and local deployment
Render configuration	Cloud deployment support
Report generation	Downloadable PDF report output

4.2 Major Modules

- Authentication module: registration, login, logout, workspace/session setup.
- Scan module: target submission, validation, scanning pipeline, and result rendering.

- Scanner modules: specialized checks for XSS, SQL injection, CSRF, directory traversal, and headers.
- Report module: report page generation and PDF export.
- Dashboard and admin module: statistics, recent scans, findings, organizations, and users.

4.3 Project Resources

GitHub Repository (provided by the team): github.com/omarnabilgaafar/aegisscan-cloud

Docker support: The project contains a Dockerfile and docker-compose.yml, which makes it valid to include a deployment/container diagram inside the report.

Docker execution evidence: the team successfully built Docker images for the project, including the final image 'aegisscan:latest', and executed the container with host-to-container port mapping. At runtime, Flask started correctly inside the container and exposed the application on localhost:5000, confirming that the system is portable and deployable through containerization.

This practical run validates the deployment setup beyond configuration files alone because it demonstrates that the provided Docker assets can launch the application successfully in a reproducible environment.

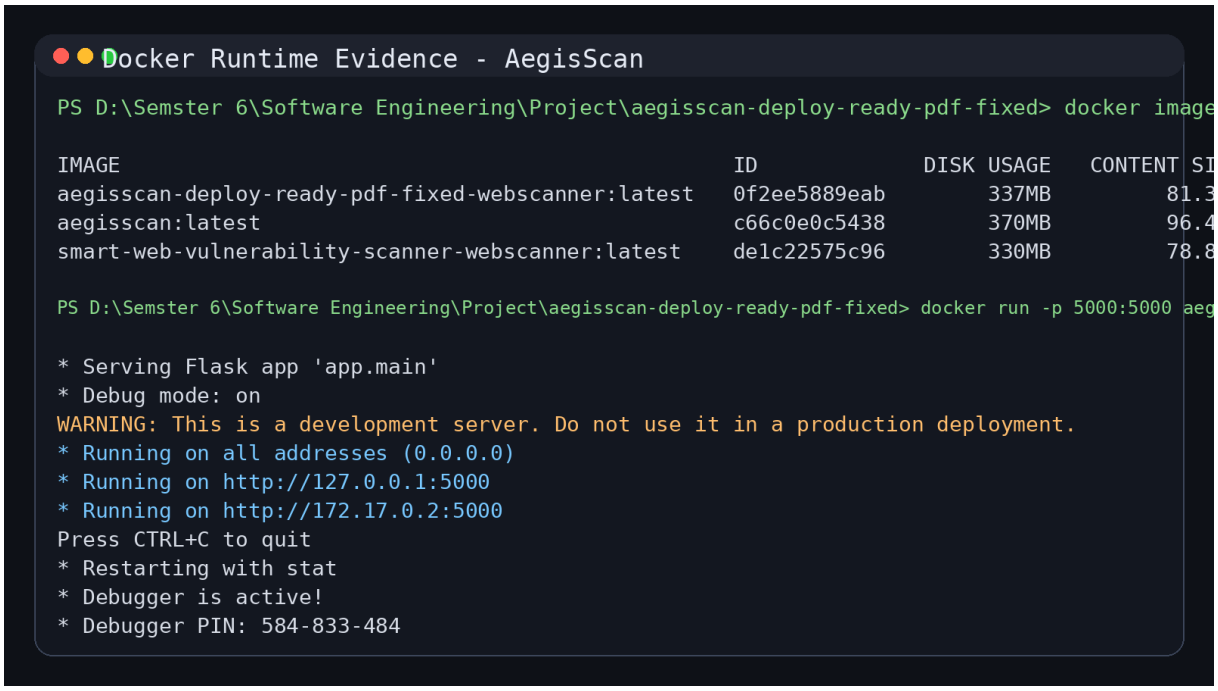


Figure 4. Docker runtime evidence: built image and successful container execution on localhost:5000.

4.4 Interface Screenshots

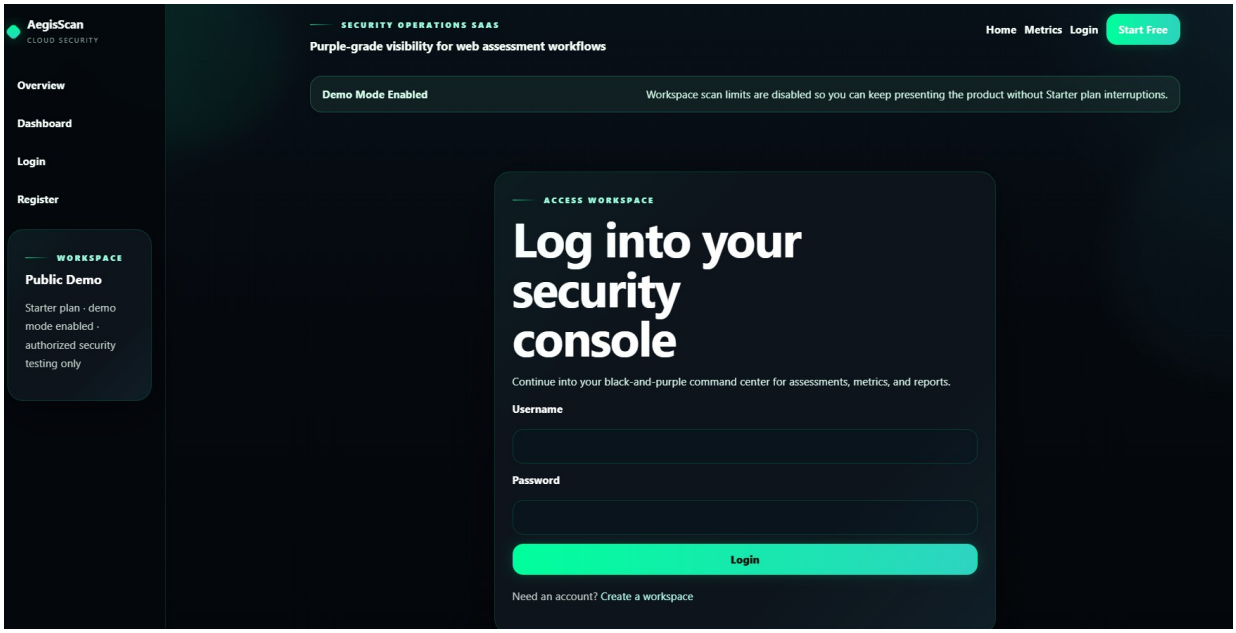


Figure 9. Login page.

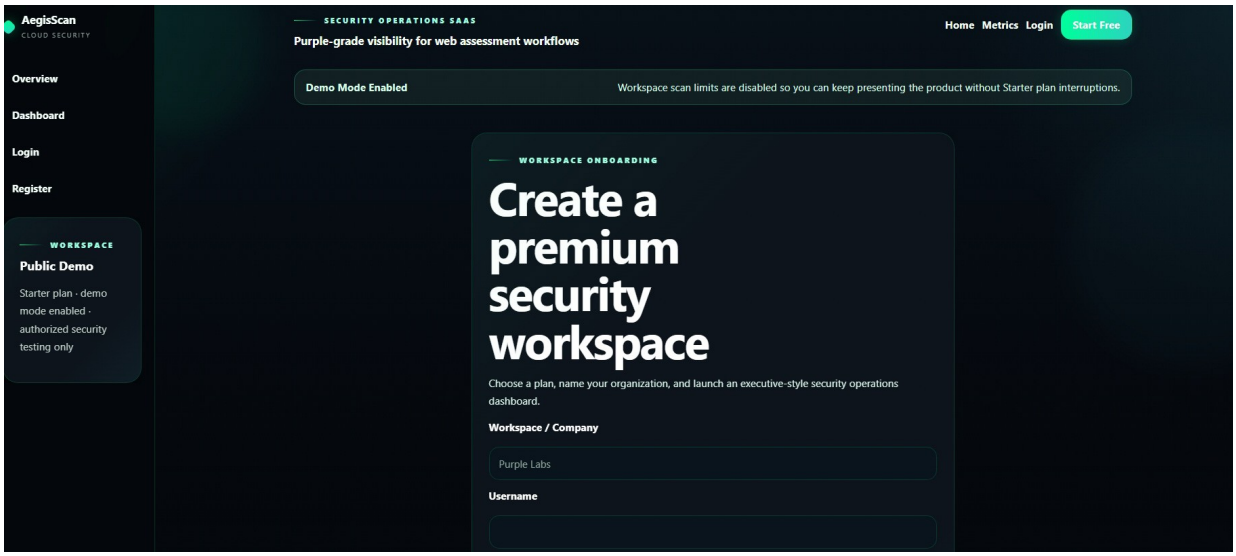


Figure 10. Registration page.

AegisScan
CLOUD SECURITY

Overview
Dashboard
Login
Register

WORKSPACE
Public Demo
Starter plan · demo mode enabled · authorized security testing only

SECURITY OPERATIONS SAAS
Purple-grade visibility for web assessment workflows

Home Metrics Login Start Free

Demo Mode Enabled Workspace scan limits are disabled so you can keep presenting the product without Starter plan interruptions.

SECURITY OPERATIONS COCKPIT

Threat monitoring dashboard

Connected to live SQLite data with workspace-aware filtering. This view is designed to make your project look like a product, not just a script with a table.

Start New Scan | Open Latest Report

Total Scans	Total Findings	Risk Score	Workspace Users	Plan
11	35	158	2	Starter · Demo Mode
stored scan sessions	all vulnerabilities in database	weighted by severity level	accounts in authentication table	Demo Workspace

Severity Distribution

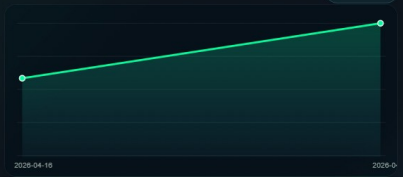
MONITORING LIVE · 35

Low	8
Medium	16
High	8
Critical	3

API KEY	USAGE	SCAN LIMIT	WORKSPACE
Not generated	0%	Unlimited (Demo Mode)	Demo Workspace

Assessment Trend

LAST 7 DAYS



Demo Mode: scan caps are bypassed while preserving SaaS plan labels.

PLAN CONSUMPTION 11 / Unlimited (Demo Mode)

Severity Donut

VISUAL SPLIT

Critical	3
High	8
Medium	16
Low	8

Issue Taxonomy

TOP CATEGORIES

Top Targets

RANKED ACTIVITY

http://127.0.0.1:5000	4
https://demo.aegisscan.local/portal	2
https://shop.aegis-demo.local	2
https://admin.aegis-demo.local	1
https://api.aegis-demo.local/v1	1

Recent Findings

LATEST

Missing Security Header
LOW

The storefront response is missing X-Frame-Options, which can increase clickjacking exposure.

https://shop.aegis-demo.local/

Missing CSRF Protection
MEDIUM

Checkout and profile forms did not show anti-CSRF controls in the simulated storefront workflow.

https://shop.aegis-demo.local/cart/checkout

Potential Stored/Reflected XSS Surface
HIGH

Review content is rendered in a customer-facing page and looks insufficiently sanitized before output.

https://shop.aegis-demo.local/reviews

Potential SQL Injection Surface
CRITICAL

Product identifiers are consumed in a query-like flow, suggesting that unsafe concatenation could expose backend data.

https://shop.aegis-demo.local/products?id=10

Potential Directory Traversal Surface
HIGH

The download handler accepts user-controlled file names and appears to lack strict normalization or allowlisting.

https://demo.aegisscan.local/portal/download?file=../../etc/passwd

Missing Security Header
LOW

Figure 11. Dashboard.

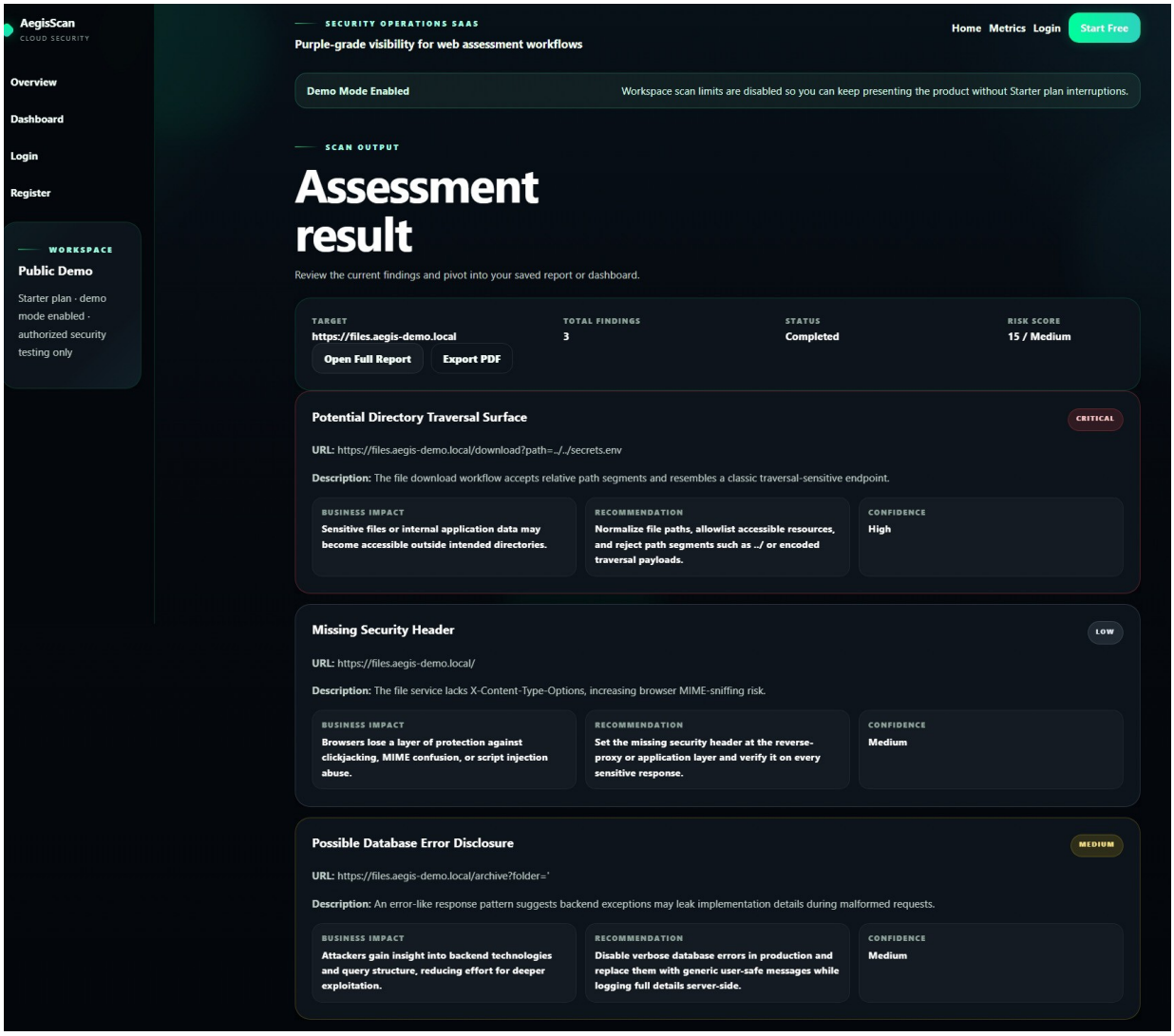


Figure 12. Scan result page.

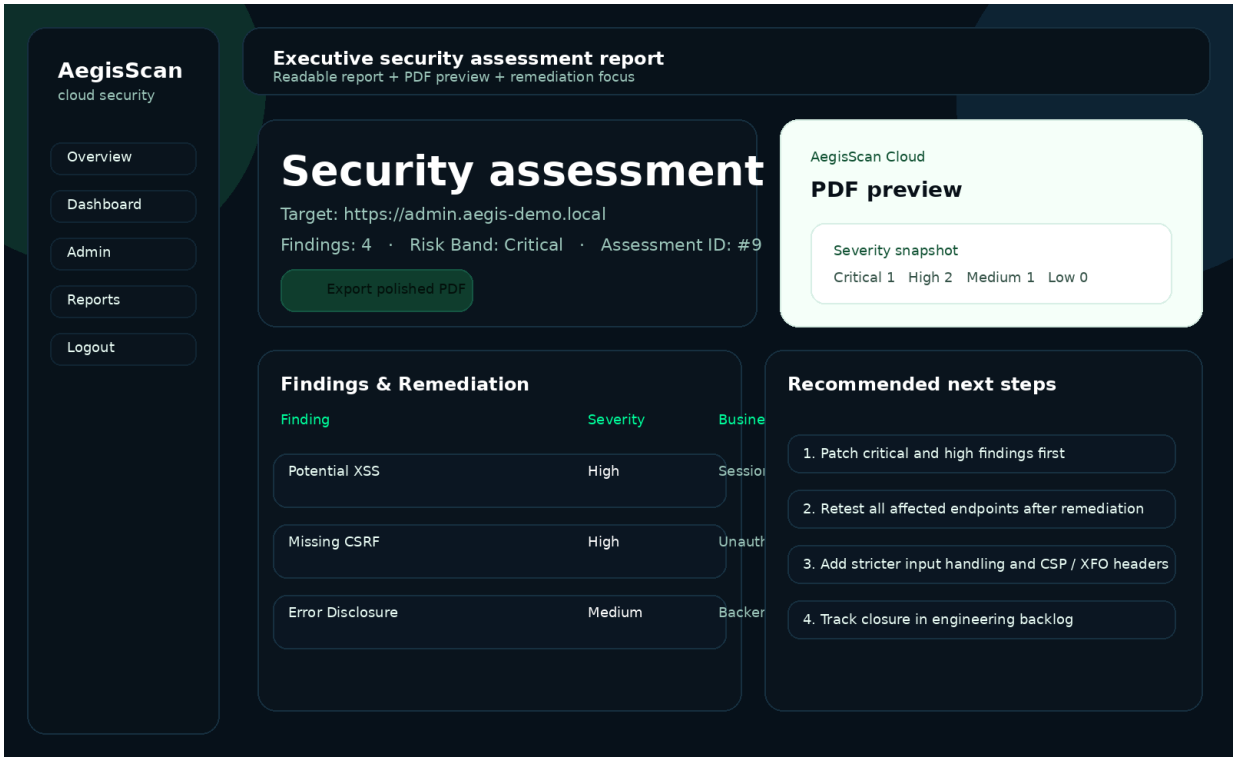


Figure 13. Report page.

5. Testing Phase

The project includes a tests directory with code-level tests that target validation, route behavior, scanner outputs, and report persistence. In this report, the testing phase is documented as a combination of unit testing, integration testing, and functional verification of the main user flow.

5.1 Testing Strategy

- Unit testing for isolated functions such as URL validation and individual scanner return structures.
- Integration testing for saving scan results and loading the home route.
- System testing for the full user flow: submit URL -> scan -> view result -> open report -> download PDF.

5.2 Sample Test Cases

ID	Test Case	Expected Result	Status
TC1	Submit empty target URL	Validation error is shown	Pass by design
TC2	Submit invalid host	Scan is rejected with safe-host message	Pass by design
TC3	Submit valid target URL	Scan result page is rendered	Pass by flow
TC4	Save scan findings	A numeric scan ID is returned	Covered by test file
TC5	Open home page	HTTP 200 response	Covered by test file

TC6	Generate report page	Findings are displayed and PDF link is available	Functional verification

6. Maintenance Phase

The modular structure makes the project maintainable. New scanners can be added as new modules, dashboard indicators can be extended, and the deployment method can be upgraded without redesigning the whole system.

6.1 Future Improvements

- Add authentication hardening, rate limiting, and audit logging.
- Add asynchronous task execution for longer scans.
- Support more vulnerability plugins and richer evidence capture.
- Use PostgreSQL or another production database for a cloud-scale version.
- Add CI/CD automation for testing and deployment.

7. Conclusion

AegisScan demonstrates the application of software engineering concepts in a real security-focused web system. The final solution includes functional requirements, design artifacts, implementation details, testing evidence, maintenance planning, UML modeling, screenshots, Docker support, and a GitHub reference suitable for project submission.